LAB REPORT

Web Application and Programming

Kanban Project Management Application

Using Django REST Framework + React JS

Name:	Abhishek Tharu
Roll Number:	PUL080BCT008
Subject:	Web Application and Programming
Date:	April 2026
GitHub:	click here

1. Project Description

This project is a full-stack Kanban Project Management application built using Django REST Framework for the backend API and React JS for the frontend. It allows teams to manage projects collaboratively using visual task boards similar to Trello. Users can create projects, organize tasks into columns, drag and drop cards between columns, set priorities and due dates, and manage their team members.

1.1 Objectives

- Build a RESTful API using Django REST Framework with JWT authentication
- Create a dynamic React frontend with drag-and-drop functionality
- Implement full CRUD operations for Projects, Columns, Cards, and Comments
- Handle user authentication and authorization using JSON Web Tokens
- Demonstrate real-world full-stack application architecture

1.2 Key Features

- User registration and login with JWT token-based authentication
- Create, edit, and delete projects with member management
- Kanban board with columns (To Do, In Progress, Done)
- Drag and drop cards between columns using @hello-pangea/dnd
- Card details: title, description, priority (low/medium/high), due date
- Card editing modal with inline updates
- Protected routes — unauthenticated users redirected to login
- Django Admin panel for backend data management
- Toast notifications for all user actions

1.3 Technology Stack

Layer	Technology	Purpose
Backend API	Django 4.x + Django REST Framework	RESTful API endpoints
Authentication	djangorestframework-simplejwt	JWT token-based auth
CORS Handling	django-cors-headers	Allow React to call Django
Frontend	React JS (Create React App)	UI and state management
HTTP Client	Axios	API calls from React
Drag & Drop	@hello-pangea/dnd	Card drag between columns
Routing	react-router-dom	Frontend page navigation
Notifications	react-hot-toast	Success/error messages
Database	SQLite (development)	Relational data storage
Admin	Django Admin	Backend data management

2. MVT Architecture

Django follows the Model-View-Template (MVT) architectural pattern, which is Django's adaptation of the classic Model-View-Controller (MVC) pattern. In this project, since we are building a REST API (not rendering HTML templates), the Template layer is replaced by React JS on the frontend. The architecture is therefore best described as Model-View-Serializer on the backend, connected to a React frontend via JSON.

2.1 MVC vs MVT Comparison

Component	MVC Role	Django MVT Role	In This Project
Model	Data layer — database structure	Model — ORM classes	Project, Column, Card, Comment models in boards/models.py
View	Business logic + response	View — handles HTTP logic	ViewSets in boards/views.py (ProjectViewSet, CardViewSet, etc.)
Template	HTML presentation layer	Template — HTML files	React JS frontend (replaces Django templates entirely)
Controller	Routes requests to views	URL Conf — urls.py	core/urls.py + boards/urls.py with DRF nested routers
Serializer	(Not in classic MVC)	Serializer — data conversion	boards/serializers.py — converts models to/from JSON

2.2 Request-Response Flow

The following describes the complete flow of a request through the application:

1. React frontend makes an HTTP request (e.g., GET /api/projects/) using Axios
2. Axios interceptor attaches the JWT access token in the Authorization header

3. Django receives the request and routes it via core/urls.py
4. The JWTAuthentication class validates the token and identifies the user
5. The request reaches the appropriate ViewSet (e.g., ProjectViewSet)
6. The ViewSet queries the database using the Django ORM (e.g., Project.objects.filter(...))
7. The queryset is passed to the ProjectSerializer which converts it to JSON
8. Django returns the JSON response with the appropriate HTTP status code
9. React receives the response, updates component state, and re-renders the UI

2.3 Application Architecture Diagram

```
[ React Frontend ] <---> [ Django REST API ] <---> [ SQLite Database ]
      |                    |
  JWT Auth Context        JWT Middleware
  Axios HTTP Client       ViewSets + Serializers
  DnD Board UI            ORM Models
```

Layer	File / Location	Responsibility
Models	boards/models.py	Project, Column, Card, Comment, User relationships
Serializers	boards/serializers.py	Convert model instances to/from JSON
Views	boards/views.py	Handle HTTP requests, apply permissions
Permissions	boards/permissions.py	IsProjectMember custom permission class
URL Routing	boards/urls.py + core/urls.py	Map URLs to ViewSets using nested routers
Auth	core/urls.py (JWT endpoints)	Register, obtain token, refresh token
Admin	boards/admin.py	Django admin registration for all models

Layer	File / Location	Responsibility
React Pages	src/pages/	Login, Register, Dashboard, Board views
React Components	src/components/	Navbar, Column, Card, CardModal
API Layer	src/api/axios.js	Axios instance with JWT interceptors
Auth State	src/context/AuthContext.jsx	Global user state, login/logout/register

3. Database Models

The application uses four main models defined in boards/models.py. All models use Django's ORM and are backed by SQLite in development.

3.1 Entity Relationship

Model	Fields	Relationships
Project	name, description, created_at	owner (FK to User), members (M2M to User)
Column	title, order	project (FK to Project) — ordered by 'order' field
Card	title, description, priority, due_date, order, created_at	column (FK to Column), assigned_to (FK to User, nullable)
Comment	text, created_at	card (FK to Card), author (FK to User)

3.2 Model Code — boards/models.py

```
from django.db import models
from django.contrib.auth.models import User


class Project(models.Model):
    name        = models.CharField(max_length=200)
    description = models.TextField(blank=True)
    owner       = models.ForeignKey(User, on_delete=models.CASCADE,
                                    related_name='owned_projects')
    members     = models.ManyToManyField(User, blank=True,
                                         related_name='projects')
    created_at  = models.DateTimeField(auto_now_add=True)


class Column(models.Model):
    project = models.ForeignKey(Project, on_delete=models.CASCADE,
                                related_name='columns')
    title   = models.CharField(max_length=100)
    order   = models.PositiveIntegerField(default=0)
    class Meta:
        ordering = ['order']
```

```
class Card(models.Model):
    PRIORITY_CHOICES = [('low','Low'),('medium','Medium'),('high','High')]
    column      = models.ForeignKey(Column, on_delete=models.CASCADE,
                                    related_name='cards')
    title       = models.CharField(max_length=200)
    description = models.TextField(blank=True)
    assigned_to = models.ForeignKey(User, on_delete=models.SET_NULL,
                                    null=True, blank=True)
    priority    = models.CharField(max_length=10,
                                   choices=PRIORITY_CHOICES, default='medium')
    due_date    = models.DateField(null=True, blank=True)
    order       = models.PositiveIntegerField(default=0)
    created_at  = models.DateTimeField(auto_now_add=True)
    class Meta:
        ordering = ['order']


class Comment(models.Model):
    card       = models.ForeignKey(Card, on_delete=models.CASCADE,
                                   related_name='comments')
    author     = models.ForeignKey(User, on_delete=models.CASCADE)
    text       = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
```

4. URL Routing

The project uses DRF Nested Routers (drf-nested-routers package) to create hierarchical REST URL patterns that reflect the data relationships. This allows clean, RESTful endpoint design.

4.1 API Endpoints

Method	Endpoint	Description	Auth Required
POST	/api/auth/register/	Register a new user	No
POST	/api/auth/token/	Login — get JWT access + refresh tokens	No
POST	/api/auth/token/refresh/	Refresh the access token	No
GET	/api/auth/me/	Get current logged-in user info	Yes
GET	/api/projects/	List all projects for current user	Yes
POST	/api/projects/	Create a new project	Yes
GET	/api/projects/{id}/	Get a single project with columns	Yes
PUT/PATCH	/api/projects/{id}/	Update project details	Yes
DELETE	/api/projects/{id}/	Delete a project	Yes
POST	/api/projects/{id}/add_member/	Add a member to project	Yes
GET/POST	/api/projects/{id}/columns/	List or create columns	Yes
PATCH	/api/projects/{id}/columns/reorder/	Reorder columns	Yes
GET/POST	/api/projects/{id}/columns/{id}/cards/	List or create cards	Yes

Method	Endpoint	Description	Auth Required
PATCH	/api/…/cards/{id}/move/	Move card to different column	Yes
GET/POST	/api/…/cards/{id}/comments/	List or add comments	Yes

4.2 URL Configuration Code

core/urls.py

```
from django.contrib import admin
from django.urls import path, include
from rest_framework_simplejwt.views import (
    TokenObtainPairView, TokenRefreshView)
from boards.views import RegisterView, MeView


urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/auth/register/', RegisterView.as_view()),
    path('api/auth/token/', TokenObtainPairView.as_view()),
    path('api/auth/token/refresh/', TokenRefreshView.as_view()),
    path('api/auth/me/', MeView.as_view()),
    path('api/', include('boards.urls')),
]
```

boards/urls.py

```
from rest_framework_nested import routers
from .views import ProjectViewSet, ColumnViewSet, CardViewSet, CommentViewSet


router = routers.DefaultRouter()
router.register(r'projects', ProjectViewSet, basename='project')


projects_router = routers.NestedDefaultRouter(
    router, r'projects', lookup='project')
projects_router.register(r'columns', ColumnViewSet, basename='project-columns')


columns_router = routers.NestedDefaultRouter(
    projects_router, r'columns', lookup='column')
columns_router.register(r'cards', CardViewSet, basename='column-cards')
```

5. Views and ViewSets

Django REST Framework ViewSets are used instead of regular Django views. ViewSets combine the logic for multiple related views (list, create, retrieve, update, destroy) into a single class. Custom actions are added using the @action decorator.

5.1 Key ViewSets — boards/views.py

Registration View

```
class RegisterView(generics.CreateAPIView):
    queryset = User.objects.all()
    serializer_class = RegisterSerializer
    permission_classes = [AllowAny]  # No auth needed to register
```

Project ViewSet

```
class ProjectViewSet(viewsets.ModelViewSet):
    serializer_class   = ProjectSerializer
    permission_classes = [IsProjectMember]

    def get_queryset(self):
        user = self.request.user
        return Project.objects.filter(owner=user).union(
               Project.objects.filter(members=user))

    def perform_create(self, serializer):
        serializer.save(owner=self.request.user)

    @action(detail=True, methods=['post'])
    def add_member(self, request, pk=None):
        project = self.get_object()
        user = User.objects.get(pk=request.data.get('user_id'))
        project.members.add(user)
        return Response({'status': 'member added'})
```

Card ViewSet — Move Action

```
class CardViewSet(viewsets.ModelViewSet):
    @action(detail=True, methods=['patch'])
    def move(self, request, pk=None, **kwargs):
        card = self.get_object()
```

```
        card.column = Column.objects.get(pk=request.data.get('column_id'))
        card.order  = request.data.get('order', 0)
        card.save()
        return Response(CardSerializer(card).data)
```

5.2 Custom Permissions — boards/permissions.py

```
class IsProjectMember(BasePermission):
    def has_object_permission(self, request, view, obj):
        if hasattr(obj, 'owner'):        # Project
            project = obj
        elif hasattr(obj, 'project'):    # Column
            project = obj.project
        elif hasattr(obj, 'column'):     # Card
            project = obj.column.project
        elif hasattr(obj, 'card'):       # Comment
            project = obj.card.column.project
        return (project.owner == request.user or
                request.user in project.members.all())
```

6. Serializers

Serializers in Django REST Framework convert complex data types (model instances, querysets) into Python native datatypes that can be rendered into JSON. They also handle deserialization — parsing incoming JSON data and validating it before saving to the database.

6.1 boards/serializers.py

```
class RegisterSerializer(serializers.ModelSerializer):
    password = serializers.CharField(write_only=True, min_length=8)
    class Meta:
        model  = User
        fields = ['username', 'email', 'password']
    def create(self, validated_data):
        return User.objects.create_user(**validated_data)
```

```
class CardSerializer(serializers.ModelSerializer):
    assigned_to    = UserSerializer(read_only=True)
    assigned_to_id = serializers.PrimaryKeyRelatedField(
        queryset=User.objects.all(), source='assigned_to',
        write_only=True, required=False, allow_null=True)
    comments = CommentSerializer(many=True, read_only=True)
    class Meta:
        model  = Card
        fields = ['id','title','description','column','assigned_to',
                  'assigned_to_id','priority','due_date','order',
                  'created_at','comments']
```

```
class ProjectSerializer(serializers.ModelSerializer):
    owner   = UserSerializer(read_only=True)
    members = UserSerializer(many=True, read_only=True)
    columns = ColumnSerializer(many=True, read_only=True)
    class Meta:
        model  = Project
        fields = ['id','name','description','owner',
                  'members','columns','created_at']
```

6.2 Serializer Concepts Demonstrated

Concept	Where Used	Purpose
ModelSerializer	All serializers	Auto-generates fields from model definition
Nested Serializer	ProjectSerializer includes ColumnSerializer	Returns full column data inside project JSON
write_only	password in RegisterSerializer	Field accepted on input but not returned in response
read_only	owner, comments fields	Field returned in response but not accepted on input
source parameter	assigned_to_id field	Maps a different field name to the model attribute
create() override	RegisterSerializer	Uses create_user() to properly hash passwords
many=True	members, columns, comments	Serializes a queryset (list) instead of single object

7. React Frontend

The frontend is built with React JS using functional components and React Hooks. It communicates with the Django API using Axios and manages global authentication state using the React Context API.

7.1 Project Structure

```
src/
├── api/
│   └── axios.js          — Axios instance with JWT interceptors
├── context/
│   └── AuthContext.jsx   — Global auth state (user, login, logout)
├── pages/
│   ├── LoginPage.jsx     — Login form page
│   ├── RegisterPage.jsx  — Registration form page
│   └── DashboardPage.jsx — Project grid dashboard
├── components/
│   ├── Navbar.jsx        — Top navigation bar
│   ├── Board/
│   │   ├── BoardView.jsx — Full Kanban board with drag-drop context
│   │   ├── Column.jsx    — Single column with droppable area
│   │   └── Card.jsx      — Draggable task card
│   └── Modals/
│       └── CardModal.jsx — Edit card details modal
├── App.jsx               — Routing + protected routes
└── index.js              — React DOM entry point
```

7.2 Authentication Flow

Authentication is handled using JWT tokens stored in localStorage. The AuthContext provides login(), register(), and logout() functions to all components.

```
const login = async (username, password) => {
  const { data } = await axios.post(BASE + 'auth/token/', {
    username, password });
  localStorage.setItem('access_token', data.access);
  localStorage.setItem('refresh_token', data.refresh);
  const me = await axios.get(BASE + 'auth/me/', {
    headers: { Authorization: `Bearer ${data.access}` }});
  setUser(me.data);
```

```
};
```

7.3 Axios Interceptor

Every Axios request automatically includes the JWT token via a request interceptor, so individual API calls do not need to manually set headers.

```
api.interceptors.request.use(config => {
  const token = localStorage.getItem('access_token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

7.4 Drag and Drop Implementation

Cards can be dragged between columns using the @hello-pangea/dnd library. The onDragEnd handler performs an optimistic UI update (updates state immediately before the API call) and then calls the card move endpoint.

```
const onDragEnd = async ({ draggableId, source, destination }) => {
  if (!destination) return;
  // 1. Update UI immediately (optimistic)
  setColumns(prev => { /* reorder logic */ });
  // 2. Sync with backend
  await api.patch(
    `projects/${projectId}/columns/${srcColId}/cards/${cardId}/move/`,
    { column_id: dstColId, order: destination.index }
  );
};
```

8. Django Admin Panel

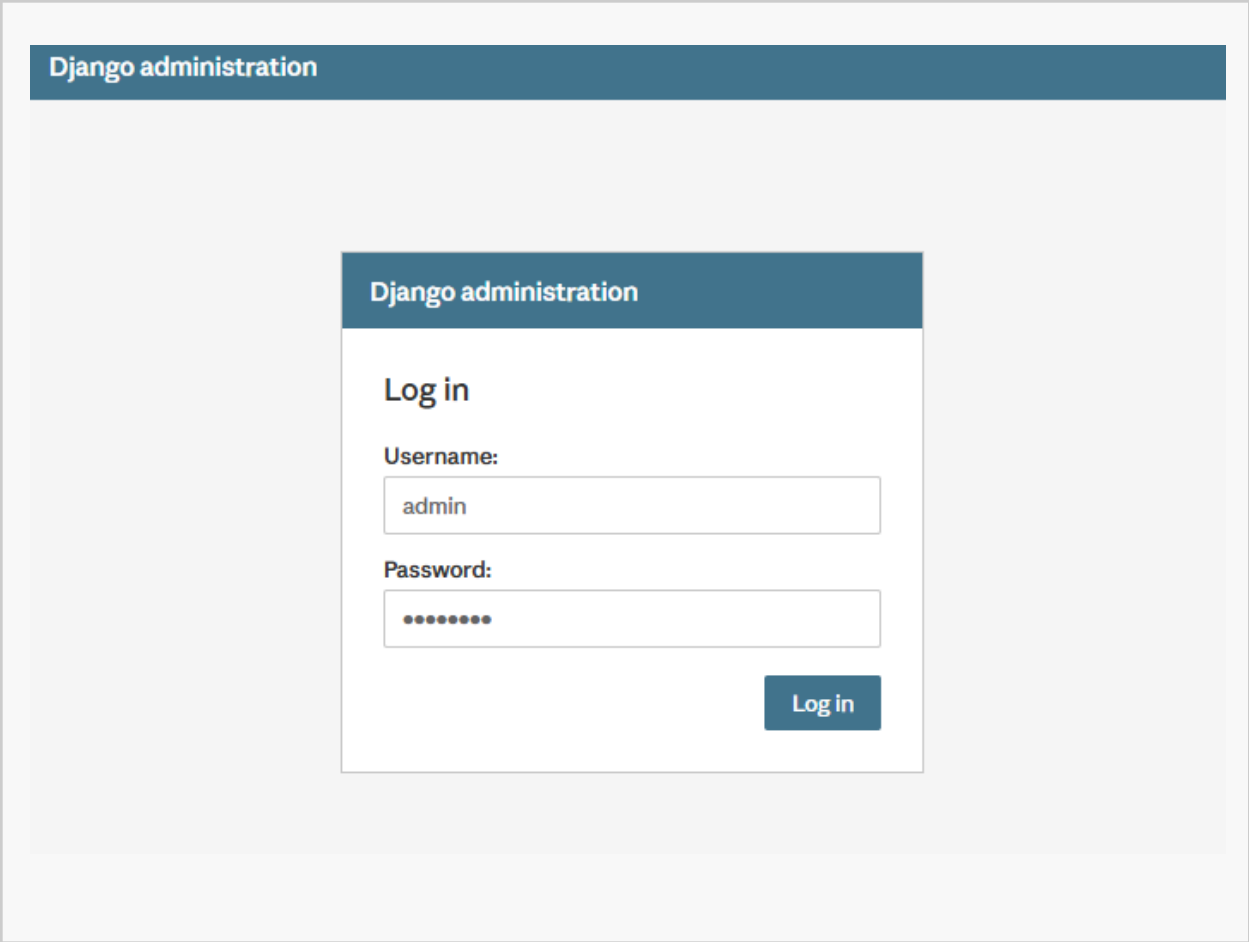
Django's built-in admin panel is configured to manage all application models. It provides a web-based interface for superusers to create, view, edit, and delete any data in the system without writing any code.

8.1 Admin Configuration — boards/admin.py

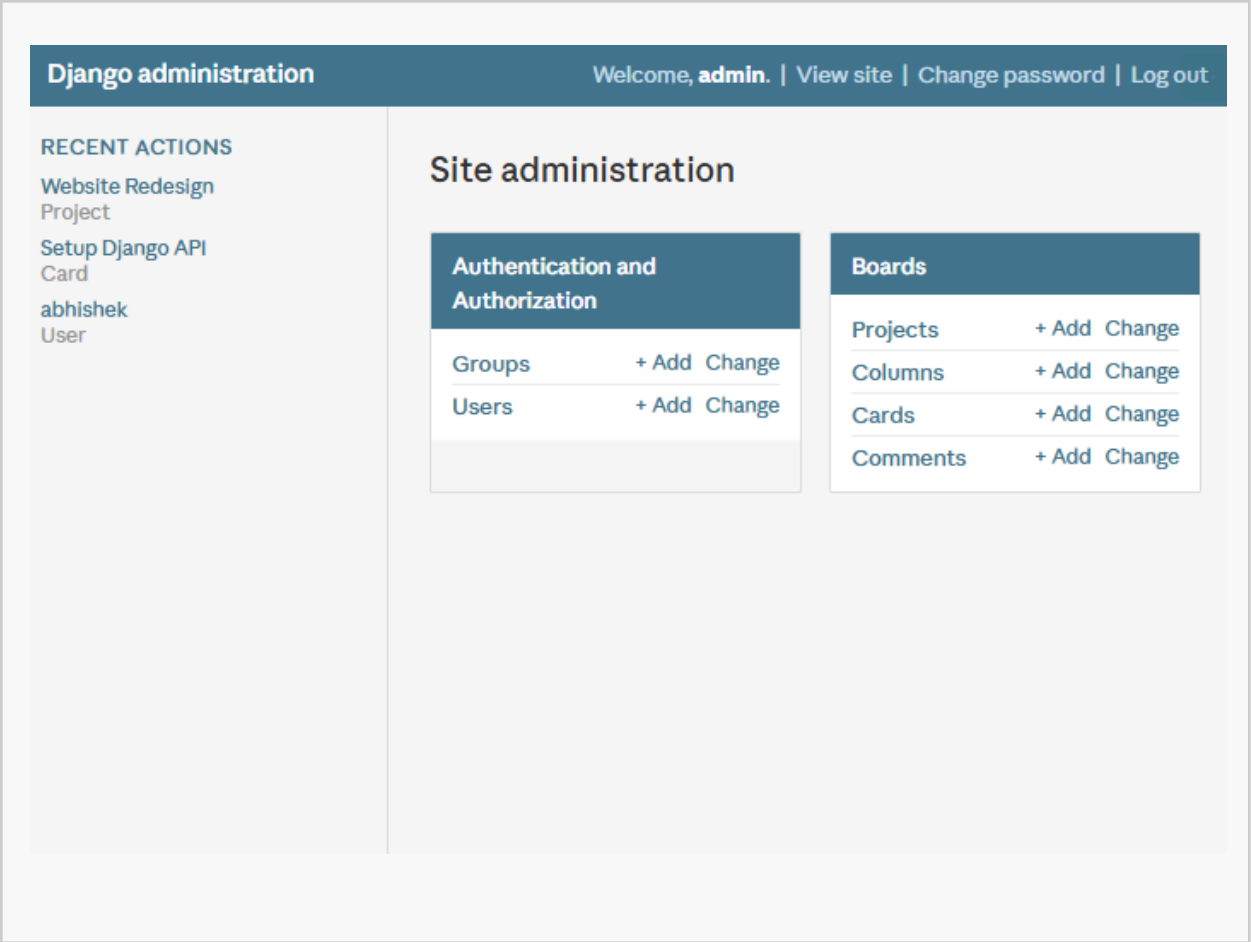
```
from django.contrib import admin
from .models import Project, Column, Card, Comment


@admin.register(Project)
class ProjectAdmin(admin.ModelAdmin):
    list_display      = ['name', 'owner', 'created_at']
    filter_horizontal = ['members']


@admin.register(Card)
class CardAdmin(admin.ModelAdmin):
    list_display = ['title', 'column', 'priority', 'due_date', 'assigned_to']
    list_filter  = ['priority']


@admin.register(Column)
class ColumnAdmin(admin.ModelAdmin):
    list_display = ['title', 'project', 'order']


admin.site.register(Comment)
```

8.2 Admin Screens

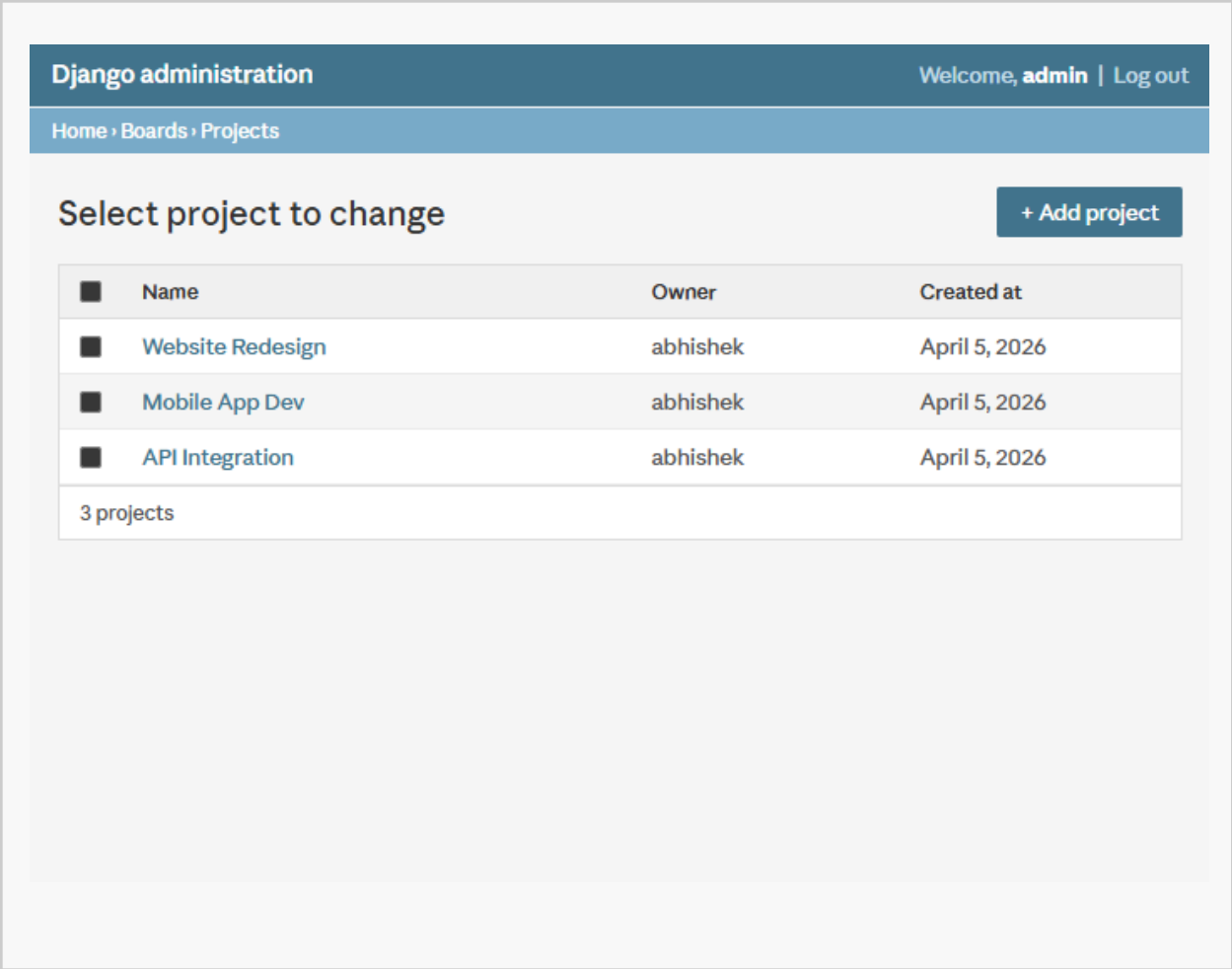
Django Admin — Login Page (http://127.0.0.1:8000/admin/)



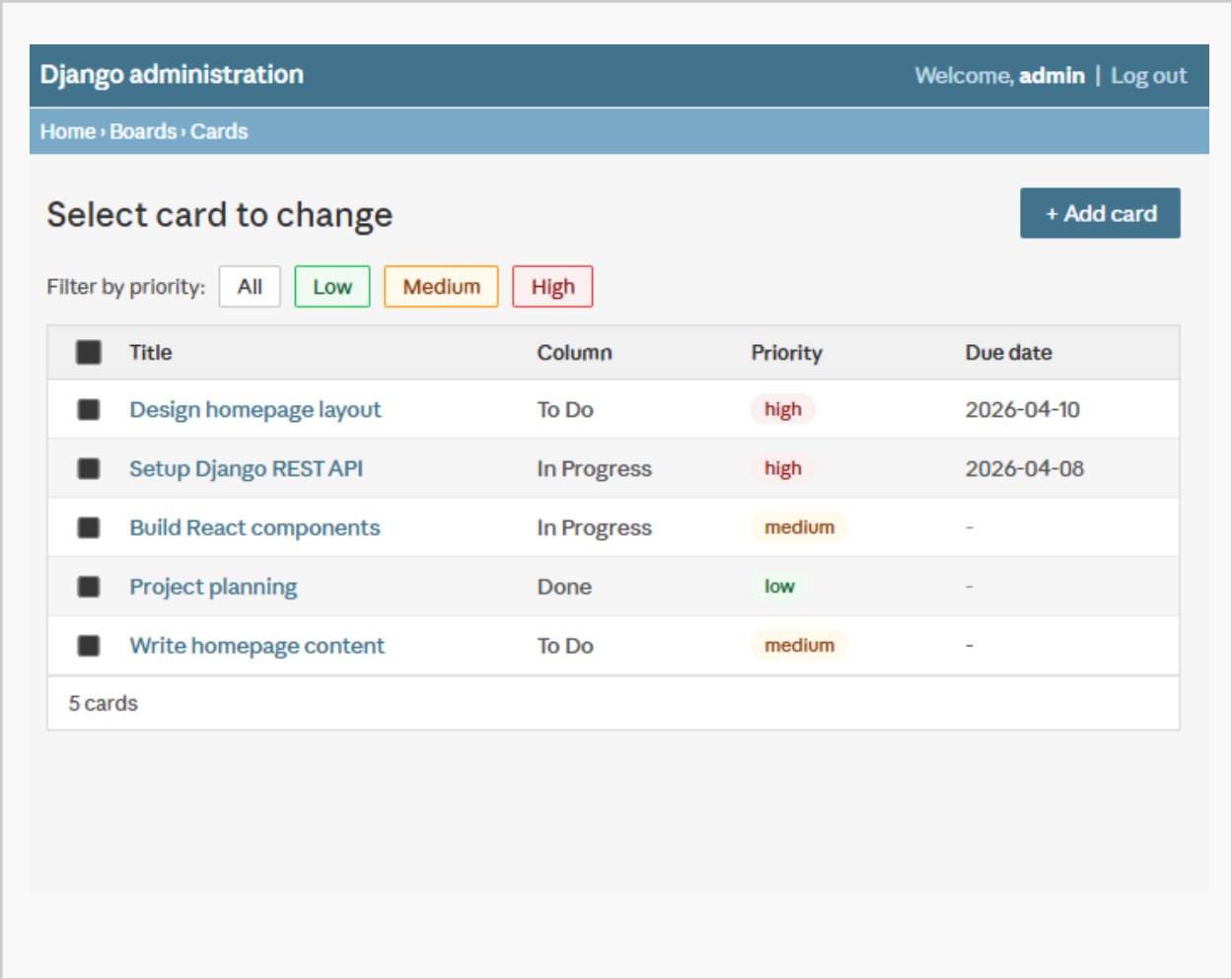
Django Admin — Home / Site Administration



Django Admin — Projects List



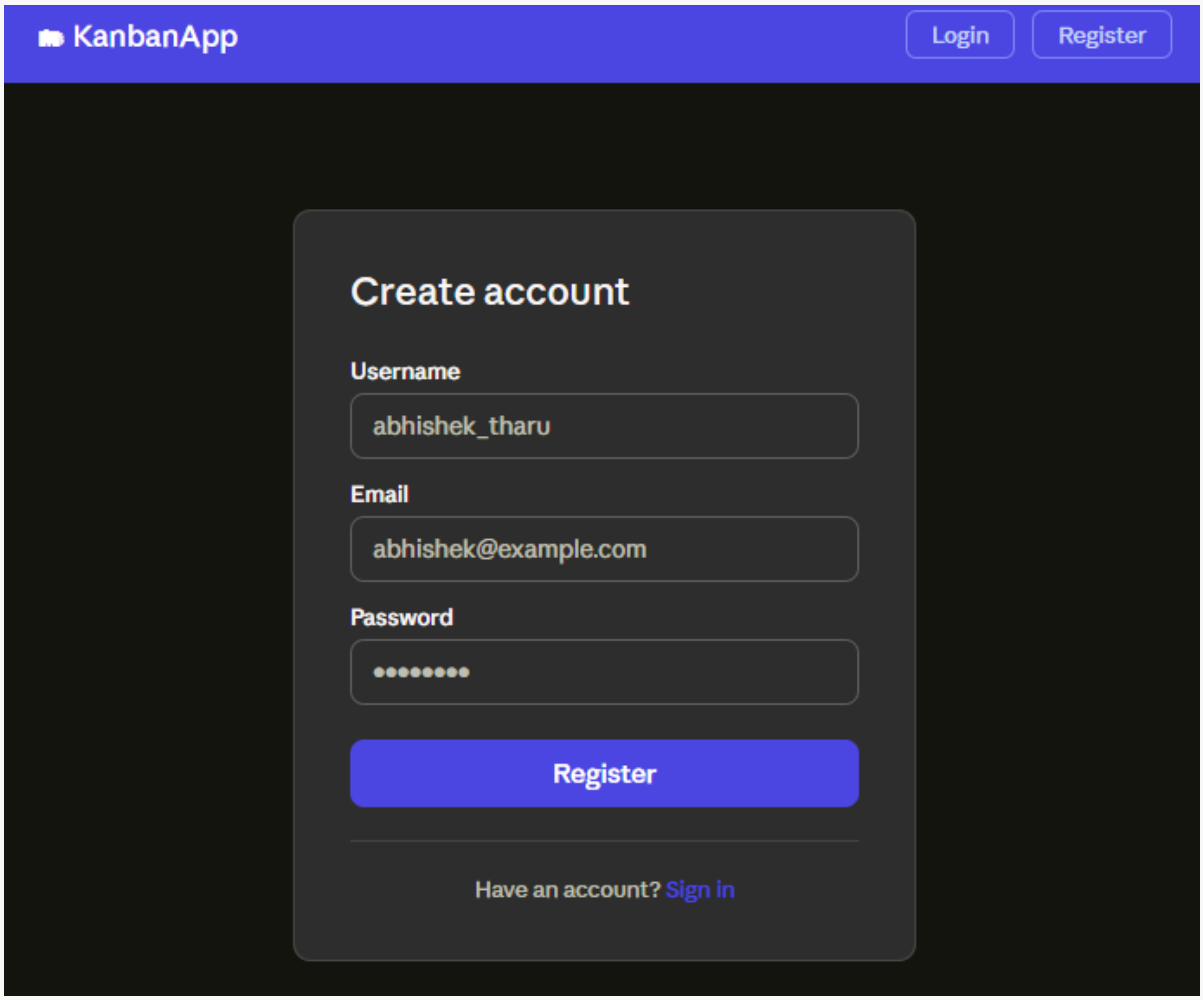
Django Admin — Cards List with Priority Filter


Django administration
Welcome, admin | Log out
Home › Boards › Cards
Select card to change
+ Add card
Filter by priority: All Low Medium High
Title
Column
Priority
Due date
Design homepage layout
To Do
high
2026-04-10
Setup Django REST API
In Progress
high
2026-04-08
Build React components
In Progress
medium
-
Project planning
Done
low
-
Write homepage content
To Do
medium
-
5 cards

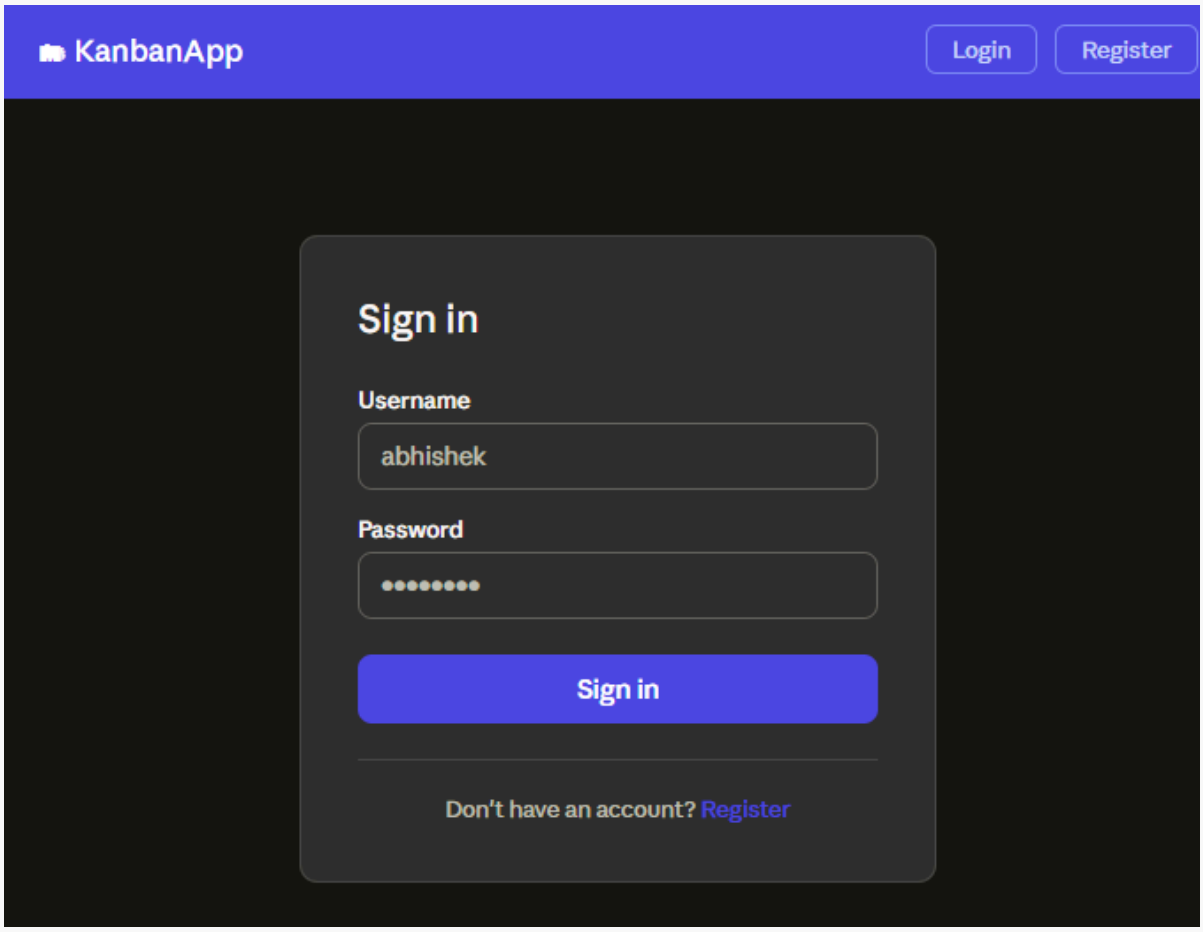
9. Output Screenshots

The following screenshots show the running application. Both servers must be running simultaneously: Django on port 8000 and React on port 3000.

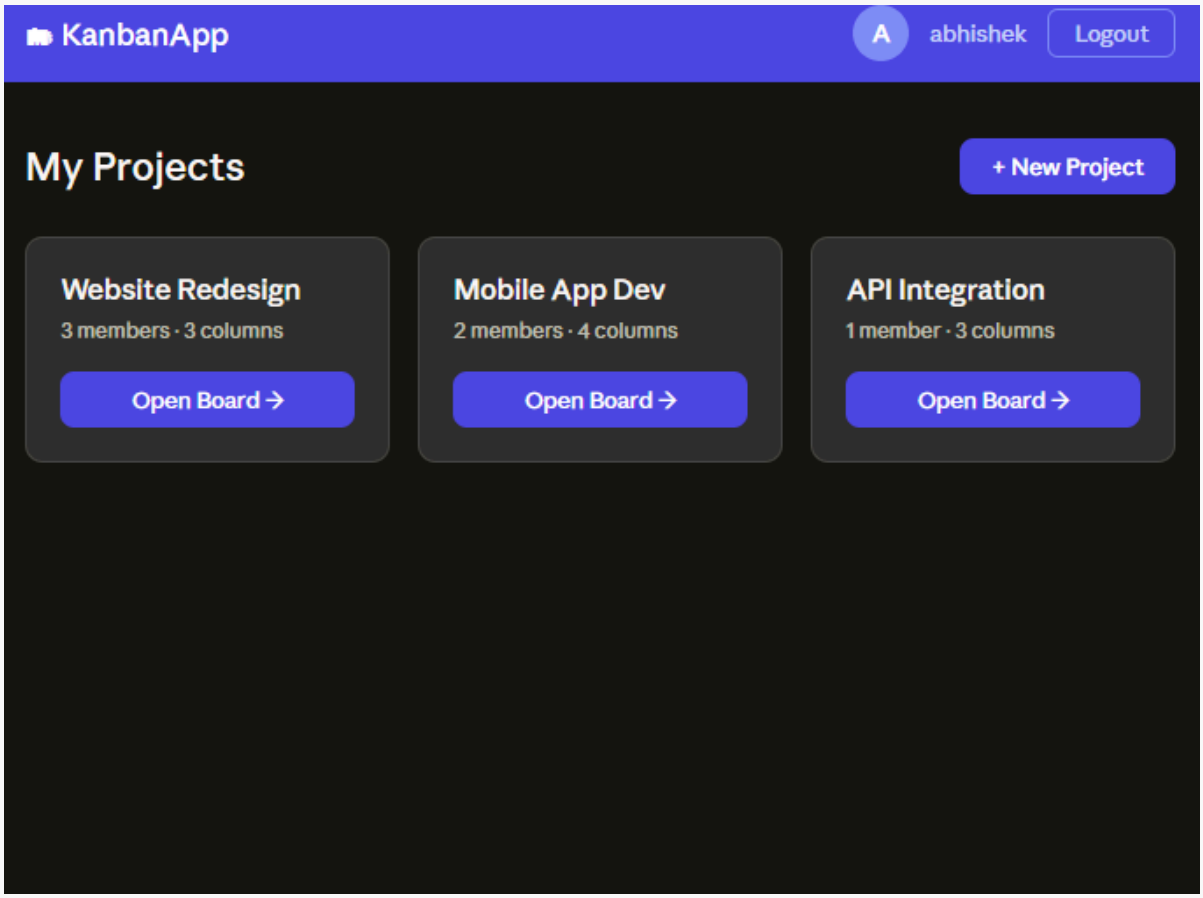
9.1 User Registration



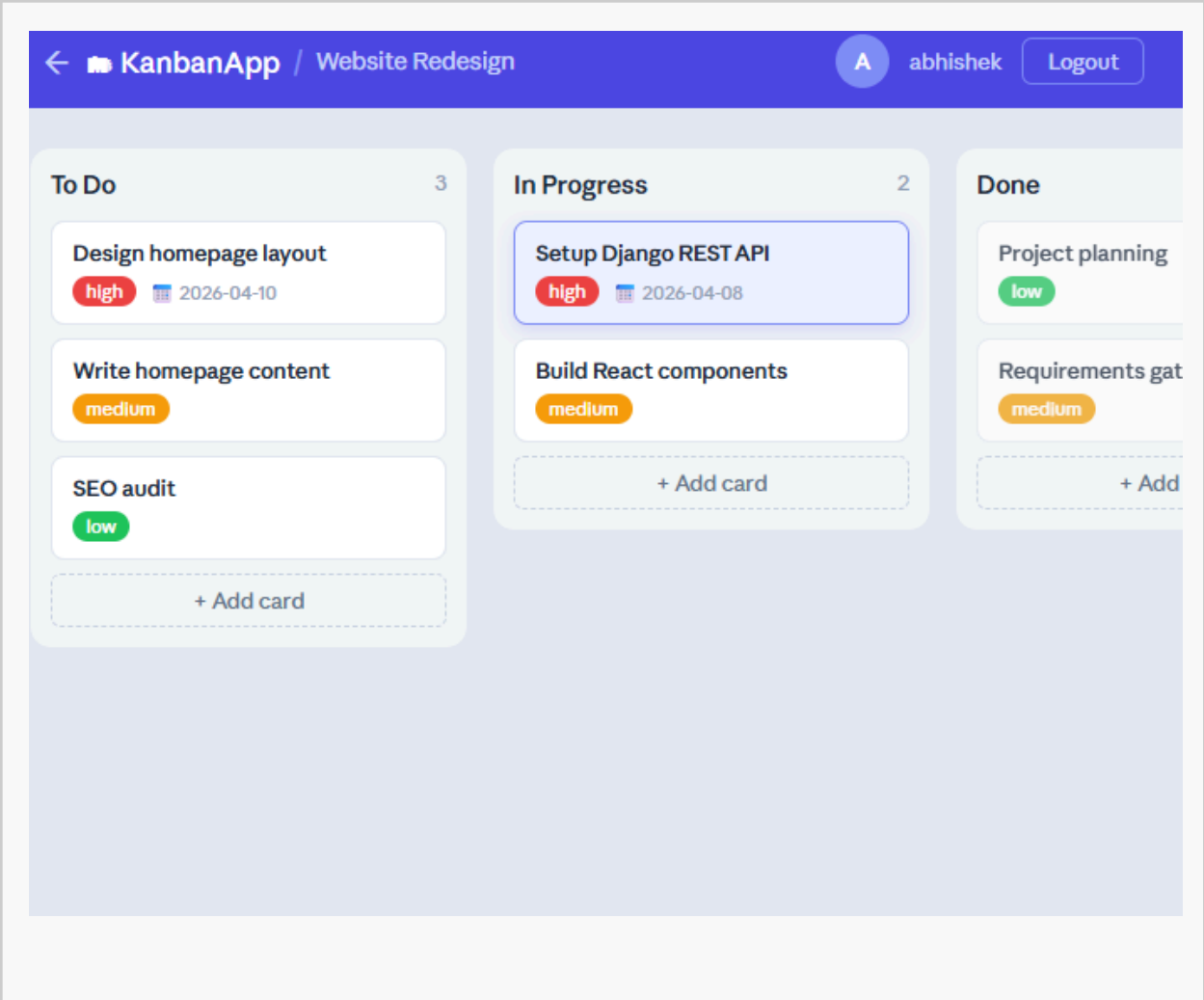
9.2 User Login



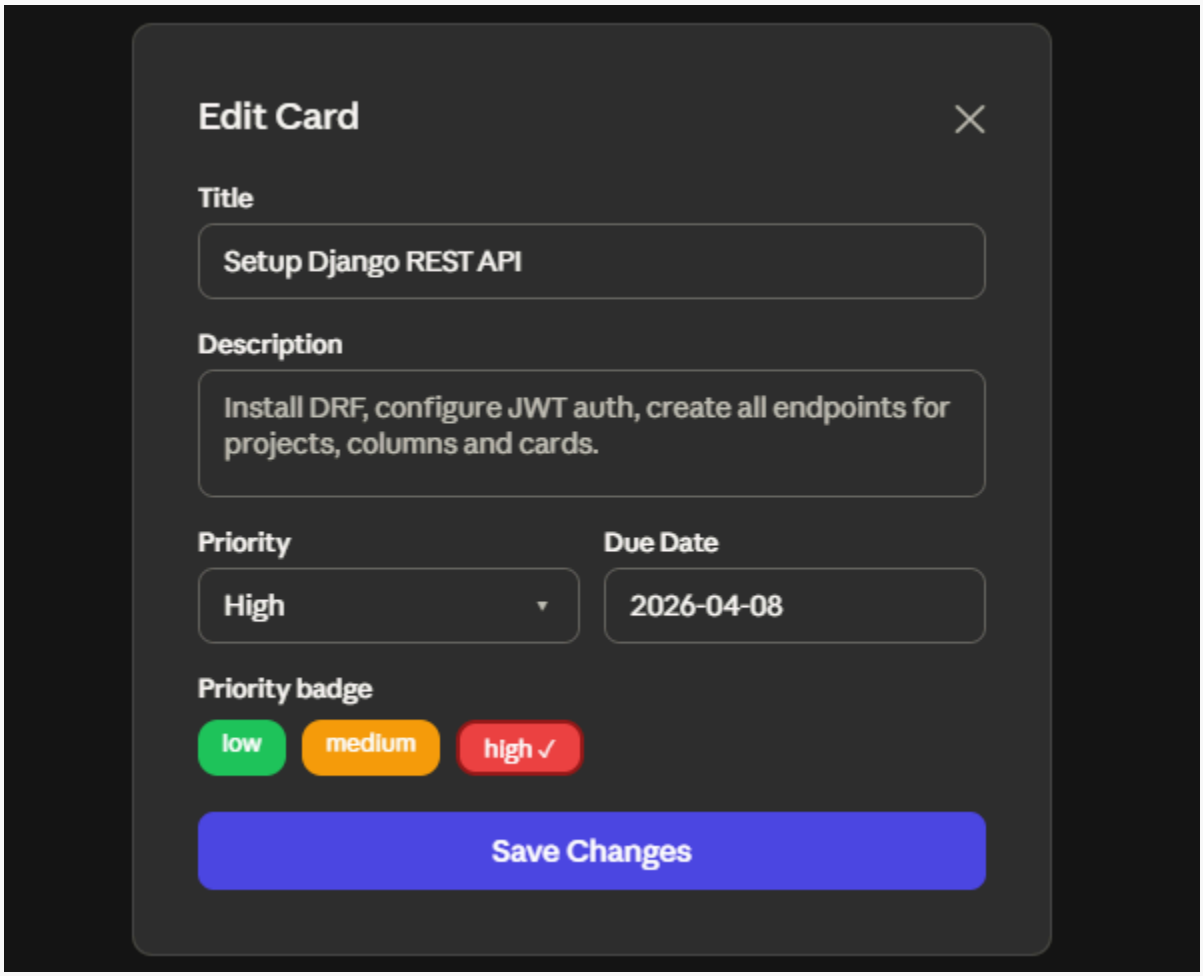
9.3 Project Dashboard



9.4 Kanban Board View



9.5 Card Edit Modal



e

9.6 Django REST Framework API

Django REST framework

Project List GET /api/projects/ 200 OK application/json

Retrieves all projects for the authenticated user.


```
[
  {
    "id": 1,
    "name": "Website Redesign",
    "owner": { "id": 1, "username": "abhishek" },
    "members": [],
    "columns": [
      { "id": 1, "title": "To Do", "order": 0 }
    ],
    "created_at": "2026-04-05T10:22:00Z"
  }
]
```


POST /api/auth/token/ 200 OK


```
{
  "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

10. Setup and Installation

10.1 Backend Setup

10. Create and activate virtual environment:

```
python -m venv venv
venv\Scripts\activate        # Windows
source venv/bin/activate     # Mac/Linux
```

11. Install Python dependencies:

```
pip install django djangorestframework djangorestframework-simplejwt
pip install django-cors-headers drf-nested-routers
```

12. Run database migrations:

```
python manage.py makemigrations
python manage.py migrate
```

13. Create superuser for admin panel:

```
python manage.py createsuperuser
```

14. Start Django development server:

```
python manage.py runserver
```

10.2 Frontend Setup

15. Create React app and install dependencies:

```
npx create-react-app kanban-frontend
cd kanban-frontend
npm install axios @hello-pangea/dnd react-router-dom
npm install jwt-decode react-hot-toast
```

16. Start React development server:

```
npm start
```

17. Open the application in browser:
    ```
    http://localhost:3000
    ```

10.3 GitHub Repository

Repository URL: github.com/YOUR-USERNAME/kanban-project

Folder	Contents
kanban_project/	Django backend — core/, boards/, manage.py
kanban_frontend/	React frontend — src/, public/, package.json
README.md	Project documentation and setup instructions

11. Key Concepts Summary

This project demonstrates the following web application development concepts:

Concept	Implementation	File / Location
REST API Design	ViewSets with CRUD + custom actions	boards/views.py
JWT Authentication	Token obtain/refresh endpoints	core/urls.py + simplejwt
Authorization	IsProjectMember permission class	boards/permissions.py
ORM — One-to-Many	Column belongs to Project (ForeignKey)	boards/models.py
ORM — Many-to-Many	Project has many members (M2M)	boards/models.py
Serializers	Nested serializers for related models	boards/serializers.py
Nested Routing	drf-nested-routers for hierarchical URLs	boards/urls.py
CORS	django-cors-headers allows React origin	core/settings.py
React Context API	Global auth state with useContext	AuthContext.jsx
Axios Interceptors	Auto-attach JWT to every request	api/axios.js
React Router	Client-side routing + protected routes	App.jsx
Drag and Drop	DragDropContext, Droppable, Draggable	BoardView, Column, Card
Optimistic Updates	Update UI before API confirms	BoardView.jsx onDragEnd
Django Admin	Registered all models for management	boards/admin.py

Concept	Implementation	File / Location
Custom Actions	@action decorator for move, add_member	boards/views.py

12. Conclusion

This lab report documents the design and implementation of a full-stack Kanban Project Management application using Django REST Framework and React JS. The project successfully demonstrates a wide range of web development concepts including RESTful API design, JWT authentication, relational database modeling with Django ORM, serialization, nested URL routing, React state management with Context API, Axios HTTP interceptors, client-side routing with protected routes, and real-time drag-and-drop interaction using @hello-pangea/dnd.

The Django backend follows the MVT architecture adapted for API development, where serializers replace traditional templates and ViewSets handle the request-response cycle efficiently. The React frontend communicates with the backend exclusively through the REST API, making the two layers completely decoupled and independently deployable.

The project can be extended in future with features such as real-time collaboration using WebSockets (Django Channels), email notifications for due date reminders, file attachments on cards, advanced filtering and search, and deployment to production using PostgreSQL, Gunicorn, and Nginx.

Key Learnings

- How to design and implement a REST API using Django REST Framework with JWT authentication
- How to model relational data using Django ORM with ForeignKey and ManyToMany relationships
- How to connect a React frontend to a Django backend using Axios with authentication
- How to implement drag-and-drop in React using the @hello-pangea/dnd library
- How to use nested routers in DRF to represent hierarchical data relationships in URLs
- How to manage global state in React using the Context API

— End of Report —